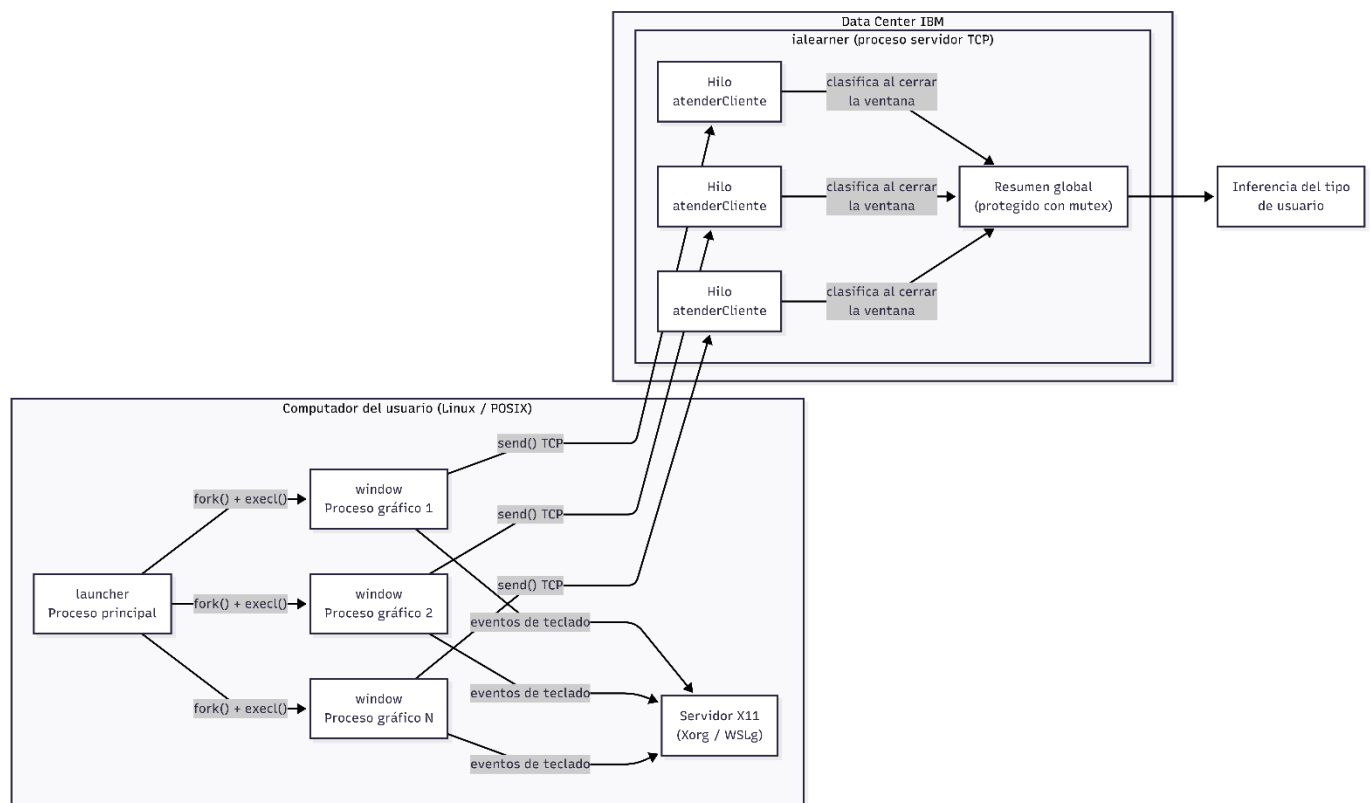




El sistema está dividido en dos partes: la computadora del usuario (sistema operativo POSIX) y data center de IBM, donde está el servidor de inteligencia artificial. Esta separación se debe al objetivo del proyecto: obtener lo que el usuario escribe localmente y enviarlo a un proceso remoto para inferir el contexto y posteriormente el tipo de usuario.

En la computadora del usuario, el proceso **launcher** actúa como punto de control: desde una consola interactiva, crea tantas ventanas gráficas como el usuario indique. Cada ventana se crea con `fork()` seguido de `execl()`, es decir, como un proceso independiente, no como un hilo. Esto se debe a que, si una ventana falla o se cierra inesperadamente, no afecta a las demás ni al launcher, algo que compartir memoria (como ocurre con hilos) no garantizaría de forma tan directa. El launcher no se queda bloqueado esperando a que las ventanas terminen; usa la señal `SIGCHLD` para enterarse de forma asíncrona cuándo una ventana se cierra, lo que le permite seguir respondiendo al menú (crear más ventanas, consultar el estado de las que ya existen, o terminarlas) en cualquier momento.

Cada ventana gráfica es cliente del servidor X11 del sistema para dibujarse y capturar el teclado, y al mismo tiempo es cliente TCP del servidor remoto **ialearner**, ubicado en el data center. Se eligió TCP porque el sistema necesita que cada letra tecleada llegue completa, en el orden correcto y sin pérdidas — justamente lo que TCP garantiza a nivel de transporte de datos.

Del lado de data center, **ialearner** es un único proceso servidor que, por cada ventana que se conecta, lanza un **hilo** independiente (con la librería `Pthreads`) encargado de recibir texto. Aquí la elección es la opuesta a la de las ventanas: como los hilos sí necesitan compartir información (por el resumen global de cuántos documentos se clasificaron en cada categoría), usar hilos permite que todos accedan a esa memoria común sin la complejidad de comunicación entre

procesos separados. Cada hilo va acumulando las palabras de su ventana hasta que la conexión se cierra (hasta que ese proceso gráfico termina), momento en el cual se calcula el vector de frecuencias de palabras (bag of words) contra los tres diccionarios del enunciado (correo electrónico, artículo científico y reporte) y se determina la clase de ese documento. Como varios hilos pueden terminar y escribir su resultado casi al mismo tiempo, el acceso a los contadores compartidos está protegido con un mutex, evitando que dos hilos lean o modifiquen esos valores de forma simultánea y produzcan un resultado inconsistente (condición de carrera).

Finalmente, cuando ya no quedan ventanas conectadas, ialearner usa la proporción acumulada de documentos por categoría para inferir el tipo de usuario (personal administrativo, personal técnico, profesor o estudiante), siguiendo la tabla de referencia del proyecto, y lo muestra en pantalla como el contexto final capturado del usuario.